GDGOC HANDS-ON AI MODULE 1 JOURNEY

project overview

today i started my first major task for the gdgoc ai module. the goal is to perform a deep-dive exploratory data analysis (eda) on a massive spotify dataset containing over 30,000 songs. im using a combination of pandas for data management and numpy for low-level mathematical operations. the plan is to move from basic cleaning to building a professional-grade analysis class.

stage 1: data setup and cleaning

first thing i did was load the data directly from the github source. after running the initial inspection, i saw the dataset has 32,833 rows. i cheecked for missing values and found only a few in track_name and artist. i checked if any column exceeded the 20% threshold mentioned in the task, but none did, so i didnt have to drop any specific columns for being empty.

for deduplication, i decided to use track_id. i realized that using track_name alone might be risky because different artists could have songs with the same title. track_id is a unique identifier provided by spotify, so it ensures i dont count the same track twice. i found 4,477 duplicates and removed them.

one of the trickier parts of stage 1 was the manual iqr calculation. the task strictly forbade using scipy, so i used np.percentile on my numeric dataframe. it was a good reminder of how to handle distributions manually.

stage 2: genre and popularity analysis

i moved into segmenting the data by genre. using .groupby() with .agg(), i calculated the mean, median, and std dev for popularity and core features.

i discovered that the pop genre has the highest variance in popularity (std dev: 24.62). ive just realized that this makes pop a "high-risk, high-reward" genre for labels. it houses both the massive global hits and a lot of tracks that dont gain any traction at all.

analyzing the top 10 prolific artists was eye-opening. david guetta came out on top with a mean popularity of 49.37. i noticed that volume doesn't always correlate with quality,some artists with fewer tracks had higher average scores than those who released a huge volume of music.

finally, i filtered for "potential hits" (popularity > 70, danceability > 0.7, etc.). only 579 tracks passed these strict rules. pop was the dominant genre here with 193 tracks, proving it’s the most "radio-ready" genre in the dataset.

stage 3: numpy deep-dive

in stage 3, i stopped using pandas and switched to pure numpy arrays. i extracted nine audio features and performed manual min-max scaling. i used the vectorized formula (x - min) / (max - min) to get everything into a

range. it felt great to do this without any python loops.

i calculated the correlation matrix using np.corrcoef(). the strongest positive correlation is between energy and loudness (0.68), while the strongest negative is between energy and

acousticness (-0.55). musically, this makes perfect sense,loud soongs are usually high-energy, and acoustic songs are generally much calmer.

then i used boolean masking to find high-energy outliers (energy > mean + 1 std). i found 4,853 tracks. i was surprised to see that these high-energy tracks are actually less popular (mean: 34.03) than the overall average (39.33). it seems listeners prefer more balanced energy levels.

stage 4: ai reflection and documentation

for the documentation stage, i used gemini to write docstrings for my functions. i prompted it to use the google-style format.

honestly, i had a small technical hurdle here. when i first used the ai-generated code for np.eye() in my correlation function, i got a typeerror because i was passing a shape tuple instead of an integer. i had to modify the code to corr_matrix.shape. it taught me that while ai is fast, i still need to understand the underlying data structures to fix these small bugs. i decided to keep the ai-generated docstrings because they are much more professional than what i would usually write manually.

BONUS TASKS

i decided to push myself further with the bonus tasks.

for bonus 1, i built an artist fingerprint function. i compared ed sheeran and justin bieber and got a cosine similarity of 0.9999. even ed sheeran and linkin park had a similarity of 0.9997. i realized that because all features are normalized between 0 and 1, the vectors are actually very close in space, though subtle differences still exist.

for bonus 2, i used numpy broadcasting to create a genre distance matrix. it was cool to see that rock and r&b are the most similar (dist: 0.824), while pop and latin are the most different (dist: 12.67). i used the broadcasting trick centroids[:, np.newaxis, :] - centroids[np.newaxis, :, :] to avoid loops entirely.

finally, for bonus 3, i refactored everything into a class called spotifyanalyzer. i used type hints like dict[str, any] and np.ndarray to make the code production-ready. the final generate_report() method gives a perfect json-ready summary of the entire dataset.

final non-technical insights

extreme energy isn't always better,tracks with very high energy are actually less popular than average ones.

pop music is the most unpredictable genre,it’s a gamble for artists and labels alike.

productivity doesn't equal popularity. quality and engagement matter more than the number of tracks an artist releases.

extra notes:

sorry for the bad formatting, it was kinda rushed since its still holiday week. although if i were to rate this module it was pretty fun and i learned quite a lot from it.